Disjoint sets: solutions

21-E01 • Partitions, successful merges and call counts

Begin with six singleton sets. Braces list members, independently of which member is the representative.

Edge processed	Partition afterwards	Sets	Successful merges so far
None	{a},{b},{c},{d},{e},{f}	6	0
ab	{a,b},{c},{d},{e},{f}	5	1
cd	{a,b},{c,d},{e},{f}	4	2
bc	{a,b,c,d},{e},{f}	3	3
ab again	{a,b,c,d},{e},{f}	3	3
ef	{a,b,c,d},{e,f}	2	4
de	{a,b,c,d,e,f}	1	5

Six UNION calls produce only five merges. The repeated ab still finds both roots, but equal roots must return without LINK.

Counting external calls gives m=6 MAKE-SET+6 UNION=12. The twelve internal FIND calls are not additional external calls. Alternatively, expanding into MAKE/FIND/successful LINK gives 6+12+5=23 basic operations, with constant same-root checks also charged. Keep these counting conventions separate.

If same-root LINK increases rank, repeated UNION(x,x) raises a singleton's rank without adding members. This violates the 2^r root-membership bound. At most n−1 **successful merges** are possible; the number of UNION calls is unrestricted.

21-E02 • Joining tails does not update every member

Each member's set pointer names its set record; FIND reads that record's representative. Members moved into another record must update set. A tail pointer only avoids walking to the end.

Operation	Append second to first: rewrites / representative	Shorter into longer: rewrites / representative
UNION(x2,x1)	1 / x2	1 / x2
UNION(x3,x2)	2 / x3	1 / x2
UNION(x4,x3)	3 / x4	1 / x2
UNION(x5,x4)	4 / x5	1 / x2
UNION(x6,x5)	5 / x6	1 / x2
Total rewrites	15	5

The left rule makes each new singleton absorb the old long list. For general n,

$1 + 2 + \cdots + (n-1) = n(n-1)/2.$

Including construction, time is $\Theta(n^2)$ despite only 2n−1 external calls.

The weighted rule retains x2 on the first tie and subsequently moves only the new singleton. This sequence has n−1 rewrites and $\Theta(n)$ time. It does not make every sequence linear: equal-size rounds in the next problem require $\Theta(n \log n)$ rewrites.

21-E03 • Charge each member for its own moves

Equal sizes retain the first set. Only listed members change their set pointers.

Merge	Members rewritten	New size	Cumulative rewrites
(a,b)	b	2	1
(c,d)	d	2	2
(e,f)	f	2	3
(g,h)	h	2	4
(a,c)	c,d	4	6
(e,g)	g,h	4	8
(a,e)	e,f,g,h	8	12

Counts for a,b,c,d,e,f,g,h are 0,1,1,2,1,2,2,3, summing to 12. Member h's representative changes h→g→e→a; the sizes after its three moves are 2→4→8.

Every move at least doubles the containing set. Charge x only when it belongs to the shorter list being moved. After j moves its set has at least 2^j members, at most n, so $j \leq \lfloor \log_2 n \rfloor$. Summing over all members bounds rewrites by $n\lfloor \log_2 n \rfloor$; external-call overhead contributes separately to m.

For $n = 2^k$, equal-size round i has $n/2^i$ merges moving 2^{i-1} members each. Each round costs n/2 and there are k rounds: $(n/2)\log_2 n$ rewrites. The per-member upper bound need not be reached by everyone: a moves zero times, h three.

21-E04 • After finding the root, follow the original path

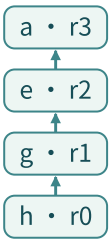
After the seven UNION calls, the root a points to itself:

Node	a	b	c	d	e	f	g	h
Rank	3	0	1	0	2	0	1	0
Parent before FIND(h)	a	a	a	c	a	e	e	g
Parent after FIND(h)	a	a	a	c	a	e	a	a

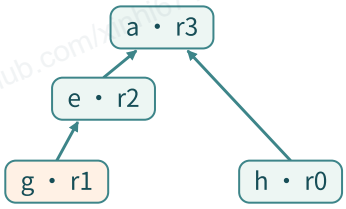
Pass one follows h→g→e→a without mutation. Pass two performs the following updates. **Save the old parent before overwriting it, so traversal can continue along the original path.**

Current x	Save old	Assign parent[x]	Next x
h	g	a	g
g	e	a	e
e	a	a, already unchanged	a
a	—	Leave root self-parent	Stop

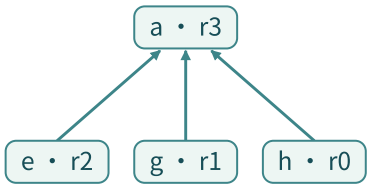
Original path



Jump to root after writing



Save old parent; finish path



If parent[h]=a is written first and traversal continues with x=parent[h], it jumps directly to a and misses g. Node e already points to a but is also skipped. The representative may still be correct; the specified full path compression has not been completed.

Nodes d,f are off this path and retain d→c→a and f→e→a. A later FIND(d) still takes two parent edges before redirecting d to a. All ranks remain unchanged. Resetting them to current depth discards the historical invariant, including strictly increasing rank on each nonroot parent edge.

This exercise keeps the first root on a rank tie; LINK in Cormen, third edition, keeps the second. Representatives and tree shapes may differ; the partition and rank bounds do not.

21-E05 • Rank records merge history, not current height

Make four singletons and perform UNION(a,b), UNION(c,d), UNION(a,c), keeping the first root on a rank tie. The first two unions give a,c rank 1; the third gives a rank 2 and leaves d→c→a. FIND(d) then makes b,c,d direct children of a.

Now a has **rank 2 and height 1**. Compression lowers height from 2 to 1 without lowering rank.

Prove the 2^r membership bound for a rank-r root by induction over operations:

1. MAKE-SET creates one member of rank 0: $1 = 2^0$.
2. Unequal-rank linking keeps the winner's rank while adding members.
3. Distinct rank-r roots represent disjoint sets of at least 2^r members each. The new rank-r+1 root therefore has at least 2^{r+1} members.
4. FIND changes neither members nor ranks, preserving the bound.

A nonroot's rank freezes when it loses root status. Its former set then contained at least 2^r distinct elements, all still among the n created elements. Thus every node satisfies $2^r \leq n$. **This historical set size is not its current subtree size after descendants move away.**

Consequently maximum rank is $\lfloor \log_2 n \rfloor$. For n=10^6, $2^{19} = 524288 \leq 10^6 < 1048576 = 2^{20}$, so it is 19. Values 0..19 give twenty possibilities; $2^4 < 20 \leq 2^5$, requiring at least five bits for rank alone. Parent indices and alignment are additional.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

21-E06 • Compute thresholds before node potentials

Function superscripts mean repeated composition. Since $A_0(j) = j + 1$, $A_0^{(i)}(j) = j + i$, hence $A_1(j) = 2j + 1$. If the i-th iterate is $2^i(j + 1) - 1$, applying $A_1(t) = 2t + 1$ gives $2^{i+1}(j + 1) - 1$. Therefore

$A_2(j) = 2^{j+1}(j + 1) - 1, \qquad A_3(1) = A_2(A_2(1)) = A_2(7) = 2047.$

Each step feeds the previous output back as input; this is not numerical exponentiation.

Fix n=16, α=3. The old path is h(0)→g(1)→e(2)→p(3)→r(4), with ranks in parentheses. For a positive-rank nonroot, parent rank determines level and the allowed iteration count; $\phi = (3 - \ell)rank - i$.

Node	Parent rank before→after	Level before→after	Iter before→after	Potential before→after
h, rank 0	1→4	Not defined	Not defined	0→0
g, rank 1	2→4	0→1	1→1	2→1
e, rank 2	3→4	0→0	1→2	5→4
p, rank 3	4→4	0→0	1→1	8→8
r, root rank 4	4→4	Not defined	Not defined	12→12

For g, new parent rank 4 crosses $A_1(1) = 3$ but not $A_2(1) = 7$: level 1, iter 1. Node e stays at level 0: $A_0(2) = 3$, $A_0^{(2)}(2) = 4$, $A_0^{(3)}(2) = 5 > 4$, so iter rises from 1 to 2.

Path potential falls 27→25. All off-path parent pointers and ranks remain unchanged, so whole-forest potential also falls by two. Nodes g,e each have a later same-level nonroot witness and are good. Node p is the last level-0 nonroot; h has rank 0 and r is the root. **A fixed node's level never falls; iter is nondecreasing only while its level stays fixed.** A level change may reset iter, as E10 shows.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

21-E07 • Paying for the path actually traversed

Actual cost is at most 2s=14. At least two good nodes lower Φ by at least one each; no other node increases it. With fixed $\Psi = 3\Phi$,

$\hat{c} = c + \Delta\Psi \le 14 + 3(-2) = 8.$

Generally suppose actual cost≤cs, choose one fixed C>c, and let $g = \max(0, s - (\alpha + 2))$. If $s \le \alpha + 2$, no potential decrease is needed: $\hat{c} \le cs \le c(\alpha + 2)$. Otherwise,

$\hat{c} \le cs - Cg = (c - C)s + C(\alpha + 2) \le C(\alpha + 2).$

Both cases give $O(1 + \alpha)$. Keep C fixed throughout the sequence, rather than increasing it whenever a long path appears.

Operation	Actual cost	Ψ before→after	ΔΨ	Amortized cost
1	3	0→4	+4	7
2	7	4→2	−2	5
3	2	2→9	+7	9
4	8	9→6	−3	5
Total	20	0→6	+6	26

Summing $\hat{c}_i = c_i + \Psi_i - \Psi_{i-1}$ cancels interior terms. Actual total is 20=26−6+0≤26 because initial potential is zero and final potential is nonnegative. Dropping coefficients and claiming $O(s)$ cancels $-\Omega(s - \alpha)$ does not establish this bound.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

21-E08 • Separate one call from a sequence

Linking and finding rules	Attainable single-FIND cost	Interpretation
Rank linking, no compression	$\Theta(\log n)$	The same long path can be traversed repeatedly
Arbitrary distinct-root linking, no compression	$\Theta(n)$	Attach each old root to a new singleton
Rank linking with full compression	$\Theta(\log n)$	m operations total $O(m(1+\alpha(n)))$

For $n = 2^k$, merge equal-rank roots in k rounds. A leaf whose containing tree loses each round reaches depth k. Construction unions find only roots and need not compress this leaf.

Without compression, f repeated finds of that leaf take $\Theta(f \log n)$, plus $\Theta(n)$ construction. With full compression, the first still takes $\Theta(\log n)$; with no intervening UNION, every later call follows one parent edge. For f≥1 the total is $\Theta(\log n + f)$, plus construction. This does not make every individual FIND worst-case constant.

The adopted definition is $\alpha(n) = \min\{k \geq 0 : A_k(1) \geq n\}$. Thresholds for k=0,1,2,3 are 2,3,7,2047. Thus for n=0,2,3,7,8,2047,2048, the values are 0,0,1,2,3,3,4.

For n≤2 there is still constant work, whereas mα(n) would be zero: retain 1+α(n) in the full theorem. At n=0 there are no valid elements for FIND or UNION; α(0)=0 is simply a function value.

21-E09 • Store application data separately from the root

Minimum keys. Forcing the smaller-key root to win can attach a high-rank root below a low-rank root. For example, make key 10 a rank-1 root and attach it to a fresh key-1 singleton of rank 0. The smaller key wins but strict parent-rank increase fails.

Keep rank linking and store minKey at each root. MAKE-SET(x) sets minKey[x]=key[x]. Before linking distinct roots, preserve both summaries; assign their minimum to the surviving root. This adds O(1) work. A query calls FIND(x) and then reads the root summary. **O(1) describes the added update, not the entire query's worst-case cost.**

Directed reachability. With only a→b, ignoring orientation merges a,b, but b cannot reach a. Strong components are {a},{b}. Equal DSU representatives certify undirected connectivity, not mutual directed reachability.

Edge deletion. If ab is the only undirected edge, deleting it should split the merged set. The original DSU has no split operation and does not retain all graph edges needed to reconstruct connectivity. Calling MAKE-SET(a) again violates its precondition rather than repairing the representation. Use a separate dynamic-connectivity or offline method.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

21-E10 Resetting iter and unsorted path levels

Thirty-two nodes: a level gain pays for resetting iter

Here $\alpha(32) = 3$. The first FIND(12) attaches rank-2 node 12 directly to rank-4 root 0. UNION(16,0) raises root 16 to rank 5, so the next find follows $12 \to 0 \to 16$, with ranks $2, 4, 5$.

Node	Before (ℓ, i, ϕ)	After (ℓ, i, ϕ)
12	$(0, 2, 4)$	$(1, 1, 3)$
0	$(0, 1, 11)$	$(0, 1, 11)$
16	Root, $\phi = 15$	Root, $\phi = 15$

The old parent rank 4 permits two A_0 iterations but does not reach $A_1(2) = 5$. The new parent rank 5 reaches level 1 and permits one A_1 iteration. Potential changes by $-1 \cdot 2 + (2 - 1) = -1$. The path potential falls from 30 to 29; all other nodes are unchanged, so the whole forest has $\Delta\Phi = -1$. Node 12 is good because later nonroot 0 has the same old level. Node 0 is the last at that level; node 16 is the root.

Sixteen nodes: different nodes can have decreasing levels

Here $\alpha(16) = 3$. The legal path is $2 \to 0 \to 8$ with ranks $1, 3, 4$. At node 2, $A_1(1) = 3 \le 3 < A_2(1) = 7$, so $(\ell, i, \phi) = (1, 1, 1)$. At node 0, $A_0(3) = 4 \le 4 < A_1(3) = 7$, so $(\ell, i, \phi) = (0, 1, 8)$. Root 8 has potential 12.

Path levels are 1 then 0. Neither nonroot has a later same-level nonroot, so $G = 0$. FIND(2) raises its parent rank from 3 to 4 without crossing its next threshold; all three potentials stay fixed. Comparing different nodes does not contradict temporal monotonicity of a fixed node's level.

A lower bound need not equal the actual count

Both paths have $s = 3$, $\alpha = 3$, hence $g = \max(0, 3 - 5) = 0$. The first has $G = 1 \ge g$, the second $G = 0 = g$. A short path need not have good nodes: when $s \le \alpha + 2$, its scan is already bounded by $cs \le c(\alpha + 2)$ without a potential release.

21-E11 A nonadjacent witness and a two-level jump

Building 0..31 leaves node 14 with rank 1,parent 12, whose rank is 2. FIND(12) attaches 12 to root 0 of rank 5. UNION(32,0) attaches 0 to root 32 of rank 6. The final union joins two rank-6 roots, raising 32 to rank 7. Thus FIND(14) follows $14 \to 12 \to 0 \to 32$, ranks 1,2,5,7.

Node	Parent rank: old→new	Level: old→new	Iter: old→new	ϕ: old→new
14	2→7	0→2	1→1	2→0
12	5→7	1→1	1→1	3→3
0	7→7	0→0	2→2	13→13
32, rank-7 root	7→7	Not defined	Not defined	21→21

For 14 afterwards, $A_2(1) = 7 \le 7 < A_3(1) = 2047$ gives level 2; $A_2^{(2)}(1) = 2047 > 7$ gives iter 1. Potential is $(3-2)1 - 1 = 0$. Both parent ranks 5,7 of node 12 lie between $A_1(2) = 5$ and $A_1^{(2)}(2) = 11$, so its level,iter,potential remain 1,1,3.

For node 0, $A_0^{(2)}(5) = 7 < A_0^{(3)}(5) = 8$, and $7 < A_1(5) = 11$: level=0,iter=2,potential=13. Root potential is $3 \cdot 7 = 21$. Path total changes $39 \to 37$. All other nodes keep rank,parent rank,root status, so the whole forest has $\Delta\Phi = -2$.

Old levels are 0,1,0. Only 14 is good, witnessed by 0. Nodes 12 and 0 are last at levels 1 and 0; the root has no level. An adjacent-only test misses 14. Actual $G = 1$, whereas $g = \max(0, 4 - (3+2)) = 0$. The decrease of two exceeds the guaranteed one.

The five steps are $7 \ge 7$; $7 \ge A_0(5) = 6$; $6 \ge A_0(2) = 3$; $3 \ge A_0(A_0^{(1)}(1)) = 3$; and $3 = A_0^{(2)}(1)$. New parent rank 7 crosses not only the next old-level threshold 3 but the level-2 threshold. A level may jump twice; a good node need not lose exactly one unit.

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study